

INSTITUTO FEDERAL CATARINENSE - IFC
BACHARELADO DE CIÊNCIA DA COMPUTAÇÃO

PATHFINDING EM MALHAS 3D : Análise estatística e comparativa

ANDRÉ VITOR BASTOS DE MACÊDO

Blumenau
2026

ANDRÉ VITOR BASTOS DE MACÊDO

PATHFINDING EM MALHAS 3D : Análise estatística e comparativa

Orientador(a): Prof. Paulo César Rodacki Gomes

SUMÁRIO

1 INTRODUÇÃO	5
2 JUSTIFICATIVA	6
3 OBJETIVOS	7
3.1 Geral	7
3.2 Específicos	7
4 REFERENCIAL TEÓRICO	9
4.1 Grafos e Topologias 3D	9
4.2 Algoritmos de Busca Heurística	9
4.3 Malhas de Navegação (NavMeshes)	9
4.4 Concorrência e Paralelismo em Sistemas de Tempo Real	9
5 METODOLOGIA	10
5.1 Infraestrutura Gráfica	10
5.1.1 Abstração de Primitivas e Malhas 3D	10
5.1.2 Gerenciamento de Recursos de Renderização	10
5.1.3 Modelo de Execução Lambda	11
5.1.4 Sistema de Callbacks e Loop de Eventos	11
5.2 Estrutura de Grafos	11
5.2.1 Abstração de Grafos	12
5.2.2 Representação de Grafos Direcionados e Não-Direcionados	12
5.2.3 Otimização de Memória com lwGraph	12
5.2.4 Algoritmos Famosos	12
5.3 Geração de Cenários de Teste	12
5.3.1 Labirintos	12
5.3.2 Mapas de Altura	13
5.3.3 Perlin Noise	14
5.3.4 NavMeshes	16
5.4 Algoritmos de Busca Heurística	16
5.4.1 Algoritmo de Dijkstra	16
5.4.2 Funções Heurísticas e Estimativas de Custo	16
5.4.3 A-Star (A*)	16
5.4.4 Jump Point Search (JPS)	16
5.4.5 Theta-Star (Theta*)	17
5.5 Multithread	17
5.5.1 Gerenciamento de Tarefas com Monitor	17
5.5.2 Testes de Estresse	17
5.6 Processo de Coleta e Tratamento de Dados	17
5.6.1 Definição de Variáveis e Métricas	17
5.6.2 Pipeline de Exportação e Análise	17
6 CRONOGRAMA	18

REFERÊNCIAS 19

1 INTRODUÇÃO

2 JUSTIFICATIVA

Com este capítulo, pretende-se justificar a relevância e a necessidade do estudo proposto, destacando a importância de uma análise estatística detalhada dos algoritmos de pathfinding em malhas 3D, especialmente considerando as complexidades e desafios dos ambientes reais.

Na área de algoritmos de pathfinding, a maioria dos estudos se concentra em topologias 2D ou não geométricas e em cenários ideais, onde as condições são controladas e otimizadas para destacar as vantagens de cada algoritmo. No entanto, a transição para ambientes 3D introduz uma série de desafios adicionais, como a complexidade da geometria, a necessidade de lidar com obstáculos tridimensionais e a gestão de recursos computacionais. A falta de análises estatísticas detalhadas sob condições adversas limita a compreensão real do desempenho desses algoritmos em situações práticas, onde otimizações matemáticas e técnicas avançadas podem ter um impacto significativo.

Este estudo se diferencia por adotar uma abordagem aprofundada e detalhada, focando em métricas quantitativas e condições adversas que refletem os desafios do mundo real. Ao invés de uma visão superficial e generalizada, a pesquisa se propõe a investigar o comportamento de algoritmos de busca quando expostos a concorrência computacional e arquiteturas complexas.

Também, a construção de uma infraestrutura gráfica e de uma biblioteca de grafos do zero não apenas proporciona um ambiente de teste personalizado, mas também garante um controle total sobre as variáveis e anomalias de hardware que podem afetar os resultados. A correta aplicação teórica e estrutural da base de grafos é fundamental para garantir a validade dos testes empíricos, conforme destacado por Gomes (2022) e Cormen et al. (2012).

Com isso, viu-se a necessidade de um estudo que vá além dos testes em ambientes ideais, oferecendo uma análise estatística robusta e detalhada do desempenho dos algoritmos de pathfinding em malhas 3D, contribuindo para a escolha informada de técnicas de navegação em projetos futuros.

3 OBJETIVOS

Neste capítulo serão apresentados os objetivos gerais e específicos do estudo, detalhando o que se pretende alcançar com a pesquisa. As metas girarão em torno da implementação de algoritmos de pathfinding, desenvolvimento de uma infraestrutura gráfica para testes, coleta e análise de dados, e apresentação dos resultados.

3.1 Geral

Perante o cenário atual de desenvolvimento de jogos e ambientes interativos, onde a navegação eficiente é crucial, este estudo visa fornecer insights valiosos sobre o desempenho de algoritmos de pathfinding em malhas 3D. Através da implementação de uma infraestrutura gráfica e da coleta de dados detalhados, pretende-se comparar algoritmos como A*, JPS e Theta* sob condições adversas, incluindo concorrência e otimizações. O objetivo é não apenas medir o tempo de execução, mas também entender as nuances do comportamento desses algoritmos em ambientes complexos.

3.2 Específicos

1. Implementar grafos: Desenvolver uma estrutura de dados leve e eficiente para representar grafos, considerando as particularidades de grafos direcionados e não direcionados. A implementação deve ser baseada em teorias e práticas recomendadas, garantindo que a estrutura seja otimizada para uso em algoritmos de pathfinding.

2. Desenvolver a infraestrutura 3D: Criar um ambiente para renderização de malhas 3D, com foco na separação de responsabilidades entre renderização gráfica e as demais aplicações. A infraestrutura deve ser capaz de gerenciar componentes, câmeras e buffers, garantindo que a renderização ocorra de forma fluida e sem travamentos, mesmo durante a execução dos algoritmos de pathfinding.

3. Implementar e adaptar os algoritmos heurísticos: Implementar os algoritmos de pathfinding A*, JPS e Theta*, adaptando-os para ambientes 3D. Otimizando-os com técnicas de alto desempenho e multithreading.

4. Adaptar a arquitetura lógica para testes de estresse e multithreading: Modificar a arquitetura do sistema para permitir a execução concorrente dos algoritmos de pathfinding, garantindo que os testes de estresse possam ser realizados sem interferências ou travamentos. A implementação deve ser baseada em práticas recomendadas para multithreading, garantindo a integridade dos dados e a eficiência do processamento.

5. Desenvolver geradores de malhas topológicas complexas: Criar geradores de malhas que possam simular diferentes cenários de teste, incluindo labirintos e topologias densas.

6. Coletar e tratar dados: Coletar dados de desempenho dos algoritmos de pathfinding, incluindo tempo de execução, uso de memória e qualidade do caminho encontrado. O tratamento dos dados deve incluir a identificação e análise de outliers, garantindo que as conclusões sejam baseadas em dados confiáveis e representativos.

7. Apresentar análises visuais e estatísticas comparativas: Desenvolver gráficos e tabelas para apresentar os resultados da análise estatística de forma clara e compreensível. As análises devem destacar as diferenças de desempenho entre

os algoritmos de pathfinding, considerando as métricas coletadas e as condições dos testes.

4 REFERENCIAL TEÓRICO

4.1 Grafos e Topologias 3D

(Gomes, 2022), (Cormen et al., 2012) e (Russell; Norvig, 2013)

4.2 Algoritmos de Busca Heurística

(Gomes, 2022) e (Russell; Norvig, 2013)

4.3 Malhas de Navegação (NavMeshes)

(Gurung, 2019)

4.4 Concorrência e Paralelismo em Sistemas de Tempo Real

5 METODOLOGIA

Aqui, o foco é detalhar a metodologia de pesquisa, explicando como os algoritmos de pathfinding e a infraestrutura gráfica serão implementados e testados. Este projeto será conduzido como uma pesquisa experimental de caráter quantitativo, onde a implementação da infraestrutura gráfica e dos algoritmos de pathfinding ocorrerá em paralelo à coleta e análise de dados. A abordagem experimental permitirá a avaliação do desempenho dos algoritmos em condições controladas, enquanto a construção da engine do zero garantirá um ambiente de teste personalizado e otimizado para as necessidades específicas deste estudo.

5.1 Infraestrutura Gráfica

O motor gráfico será construído utilizando a biblioteca OpenGL, conforme a documentação de Vries (2014), que é amplamente utilizada para renderização 3D. A arquitetura do sistema priorizará renderizações simples e eficientes, uma vez que o foco principal é a análise de algoritmos de pathfinding e não a complexidade gráfica.

Foi priorizado o desenvolvimento de uma estrutura que escondesse as chamadas complexas do OpenGL, criando uma camada de abstração para simplificar o uso e a implementação de funcionalidades gráficas. Esta abordagem visa facilitar o desenvolvimento e a manutenção do código, permitindo que os testes de pathfinding sejam realizados sem a necessidade de lidar diretamente com as complexidades do OpenGL.

5.1.1 Abstração de Primitivas e Malhas 3D

Foi escolhido o paradigma de orientação a objetos para a implementação da infraestrutura gráfica, visando uma estrutura modular e reutilizável. A criação de classes para representar primitivas gráficas, como vértices, arestas e faces, permitirá uma manipulação mais intuitiva e eficiente das malhas 3D.

O uso de classes permite a criação de hierarquias e a aplicação do padrão de design Composite, facilitando a construção de estruturas complexas a partir de componentes mais simples. Isso é especialmente útil na criação de malhas 3D, onde diferentes elementos podem ser combinados para formar estruturas mais complexas.

As malhas 3D serão criadas utilizando uma estrutura de árvore com Composite (Gamma et al., 1994), onde cada nó representa um componente da malha, como um conjunto de vértices e índices ou outra árvore. Esta abordagem permite uma organização hierárquica dos elementos da malha, facilitando a manipulação e a renderização eficiente. A estrutura de árvore também é benéfica para a implementação de algoritmos de pathfinding, pois permite uma navegação eficiente pelos elementos da malha.

5.1.2 Gerenciamento de Recursos de Renderização

No OpenGL, a base de uma malha é composta por vértices e índices, que são armazenados em buffers de vértices (VBOs) e buffers de índices (EBOs). Para otimizar o desempenho, a infraestrutura gráfica implementará um sistema de gerenciamento de recursos que permitirá a criação, atualização e exclusão eficiente desses buffers. O sistema de gerenciamento de recursos garantirá que os buffers sejam reutilizados

sempre que possível, minimizando a necessidade de criar novos buffers e reduzindo o overhead associado à criação e destruição de recursos gráficos.

Com isso garantimos o mínimo de overhead possível durante a renderização, permitindo que os testes de pathfinding sejam realizados de forma fluida e sem interrupções, mesmo em cenários complexos.

5.1.3 Modelo de Execução Lambda

O contexto de renderização necessita de um loop de eventos para processar as interações do usuário e atualizar a renderização. Para simplificar o desenvolvimento e a manutenção do código, foi adotado um modelo de execução baseado em lambdas, onde as funções de renderização e atualização são definidas como lambdas que podem ser facilmente passadas como argumentos para o loop de eventos.

Este modelo de execução permite uma maior flexibilidade e modularidade, permitindo que diferentes partes do código sejam facilmente substituídas ou modificadas sem afetar a estrutura geral do sistema. A utilização de lambdas também facilita a implementação de callbacks para eventos específicos, como a atualização da posição da câmera ou IO do usuário, garantindo que o sistema seja responsivo e eficiente durante a execução dos testes de pathfinding.

A abstração dá total liberdade para a implementação de diferentes tipos de renderização e atualização, permitindo que o sistema seja adaptado para diferentes cenários e necessidades, sem a necessidade de modificar a estrutura subjacente do loop de eventos ou ter que lidar com a complexidade do OpenGL.

Para isso, também, foram implementados diferentes componentes de renderização (renderer, keybindings, câmeras, etc.) que são integrados ao loop de eventos. Cada componente é responsável por uma parte específica da renderização ou da interação do usuário, e o modelo de execução baseado em lambdas permite que esses componentes sejam facilmente integrados e gerenciados dentro do loop de eventos.

5.1.4 Sistema de Callbacks e Loop de Eventos

Um dos principais componentes do sistema é o responsável pelos atalhos de teclado, que permite a interação do usuário com a renderização. Este componente é implementado utilizando um sistema de callbacks, onde as funções de callback são definidas para responder a eventos específicos, como pressionar uma tecla ou mover o mouse. Como o OpenGL não possui um gerenciador de atalhos nativo, mas contém um sistema de IO, a implementação de um sistema de callbacks personalizado é essencial para fugir de complexidades adicionais durante o estudo.

O sistema de atalhos é integrado ao loop de eventos a partir de funções lambda, permitindo que as ações do usuário sejam processadas independente do nível de paralelização. Isso garante que a renderização seja atualizada em tempo real sem interrupções.

5.2 Estrutura de Grafos

Desenvolvida com base no livro de (Gomes, 2022). Rigor teórico e aplicação com base nos estudos.

Implementado visando alta escalabilidade e fácil utilização.

Futuras adaptações visando desempenho.

5.2.1 Abstração de Grafos

Teoria de grafos de (Gomes, 2022).
Programação orientada a objetos em C++ com metaprogramação.
Controle de memória.

5.2.2 Representação de Grafos Direcionados e Não-Direcionados

Classe comum. Composite pattern.
Grafos direcionados e não-direcionados.

5.2.3 Otimização de Memória com lwGraph

Utilização mínima de memória.
Fácil acesso usando inteiros.

5.2.4 Algoritmos Famosos

Algoritmos tirados do livro de (Gomes, 2022).

5.3 Geração de Cenários de Teste

Durante a fase de desenvolvimento, foram criados diversos cenários de teste para avaliar o desempenho dos algoritmos de pathfinding. É importante retificar como estes cenários foram gerados e o que eles representam em termos de desafios para os algoritmos. A seguir, serão detalhados os tipos de cenários utilizados:

5.3.1 Labirintos

Como testes iniciais, foram gerados labirintos utilizando o Algoritmo de Kruskal, que é um método eficiente para criar labirintos. Este tipo de teste é ótimo para avaliar a capacidade dos algoritmos de pathfinding em encontrar caminhos em ambientes complexos e com muitos obstáculos, forçando-os a realizar cálculos exaustivos para encontrar a rota mais eficiente.

Para criar um labirinto, segundo Buck (2011), se cria um grafo em grid com valores de peso aleatórios entre os vértices e seus vizinhos (considerando os 8 vizinhos para permitir movimento diagonal). Em seguida, é aplicado o Algoritmo de Kruskal para gerar um labirinto perfeito, uma vez que ele gera uma Árvore Geradora Mínima (MST), e criar um novo grafo a partir dessa árvore. Isso garante que para qualquer par de vértices, existe exatamente um caminho entre eles, o que é ideal para testar a eficiência dos algoritmos de pathfinding.

Consequentemente, é possível retirar ou adicionar arestas para criar variações de labirintos, aumentando a complexidade e a densidade de obstáculos, o que permitirá uma análise mais robusta do desempenho dos algoritmos em diferentes cenários.

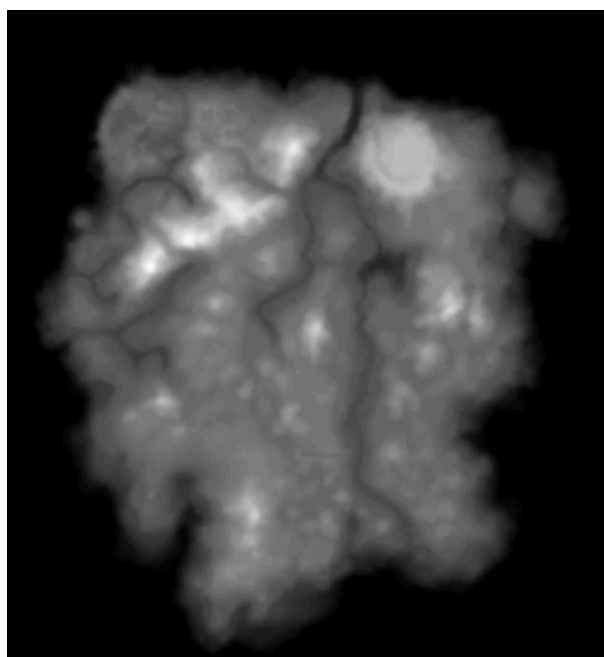
Embora os labirintos sejam um excelente teste para avaliar a capacidade de desvio de obstáculos, eles foram utilizados apenas como um ponto de partida para a geração de cenários de teste. Uma vez que a ideia desta pesquisa é comparar algoritmos de pathfindings em grafos criados a partir de malhas 3D. Gerar um grafo

abstrato e independente seria redundante a esse estudo, o que levou ao descarte deste meio de geração de cenários.

5.3.2 Mapas de Altura

Um heightmap (ou mapa de altura) é uma imagem que utiliza apenas tons de cinza para representar a elevação de uma superfície. A intensidade de cada pixel indica a altura de um determinado ponto no terreno, onde tons mais claros representam áreas mais elevadas e tons mais escuros indicam áreas mais baixas.

Figura 1 – Exemplo de Heightmap



Fonte: The Imperial Library (2026).

Determinando o valor mínimo e máximo de altura, é possível criar uma malha 3D a partir do heightmap, onde cada pixel é convertido em um vértice com coordenadas (x, y, z), sendo x e y as coordenadas do pixel e z o valor do pixel. Como trabalhamos com valores cinza, os valores RGB serão iguais, alternando apenas na intensidade entre 0 e 255. A partir disso, é possível criar uma malha 3D que representa o terreno que desejamos simular.

Com o uso dos heightmaps, é possível gerar tanto malhas 3D quanto grafos a partir de um mesmo cenário, simplificando o processo de geração de cenários. Esta versatilidade se provará útil futuramente quando falarmos de NavMeshes.

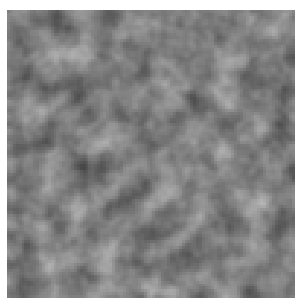
Agora, considerando que certas diferenças de altura podem ser intransponíveis para um agente navegando, foi separada a geração de malhas e grafos. Enquanto as malhas são geradas indiscriminadamente, os grafos verificam as diferenças de altura entre os vértices e, caso a diferença seja maior que um determinado limiar, a aresta entre esses vértices não é adicionada. Isso simula a realidade de que certos terrenos podem ser intransponíveis para um agente, forçando os algoritmos de pathfinding a encontrar rotas alternativas.

5.3.3 Perlin Noise

O Perlin Noise é um algoritmo de geração de ruído procedural que é amplamente utilizado para criar texturas e terrenos realistas em gráficos 3D. Ele foi desenvolvido por Ken Perlin em 1983 e é conhecido por produzir padrões de ruído suave e natural, o que o torna ideal para simular superfícies como montanhas, nuvens e oceanos. Com isso em mente, podemos utilizar desse algoritmo para gerar nossos próprios heightmaps, sem precisar recorrer a imagens pré-existentes. Isso nos dá controle total sobre a geração dos cenários de teste, permitindo criar uma variedade de terrenos com diferentes características e desafios para os algoritmos de pathfinding.

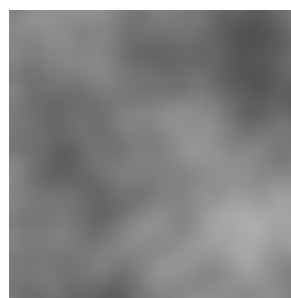
Segundo os estudos de Zipped (2023), este algoritmo funciona gerando uma grade de tamanho uniforme, onde cada ponto da grade é associado a um vetor de gradiente aleatório. Um quadrado da grade é uma área afetada por 4 vértices que futuramente terão seus vetores atribuídos. Quanto maior a área deste quadrado, mais suave será o ruído, enquanto quadrados menores produzem ruídos mais detalhados com alta frequência.

Figura 2 – Noise com grade de tamanho 10



Fonte: Elaborado pelo autor.

Figura 3 – Noise com grade de tamanho 50



Fonte: Elaborado pelo autor.

Para atribuir um valor de elevação para cada pixel do heightmap gerado pelo Perlin Noise, primeiro é gerado um vetor unitário aleatório para cada vértice da grade. Depois para cada pixel do heightmap, é calculada a contribuição de cada um dos 4 vértices do quadrado correspondente. O valor de distância entre o pixel e cada vértice de seu quadrado é calculado, em seguida o *dot product* entre o vetor de gradiente do vértice e o vetor de distância é computado. Esses valores são, então, interpolados usando uma função de suavização para garantir que o ruído seja suave e natural. O resultado final é um valor de elevação para cada pixel dessa camada do heightmap.

Patel (2015) enfatiza a importância do processo ser repetido para múltiplas camadas (octaves) de ruído, onde cada camada tem uma frequência e amplitude diferentes. A frequência tende a influenciar o nível de detalhe do ruído, enquanto a amplitude controla a intensidade das variações.

Figura 4 – Noise de tamanho 25 com 3 octaves

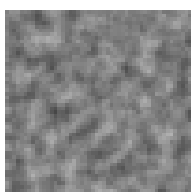


Este noise contém 3 camadas de ruído, 1 de frequência, 1 de amplitude e 1 de expoente.

Fonte: Elaborado pelo autor.

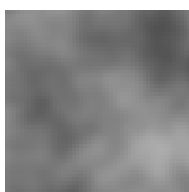
Com este mesmo noise, é possível alterar a frequência e amplitude de cada camada para criar diferentes tipos de terrenos. Por exemplo, aumentando a frequência e diminuindo a amplitude, é possível criar um terreno mais acidentado e detalhado, enquanto diminuindo a frequência e aumentando a amplitude, pode-se criar um terreno mais suave e ondulado. Também, é possível aplicar um expoente para controlar a distribuição dos valores de elevação, o que pode resultar em terrenos mais acidentados, enfatizando valores altos e baixos.

Figura 5 – Frequência 5



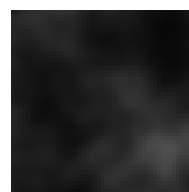
Fonte: Elaborado pelo autor.

Figura 6 – Amplitude 5



Fonte: Elaborado pelo autor.

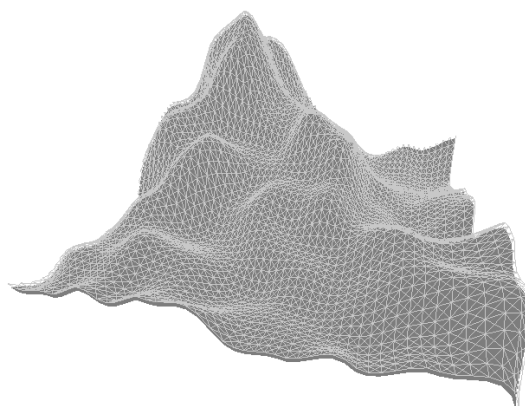
Figura 7 – Expoente 3



Fonte: Elaborado pelo autor.

Com isso, é possível criar uma variedade de cenários de teste para os algoritmos de pathfinding, desde terrenos relativamente planos até ambientes montanhosos e cheios de obstáculos.

Figura 8 – Exemplo de malha gerada com Perlin Noise



Fonte: Elaborado pelo autor.

5.3.4 NavMeshes

NavMeshes ou malhas de navegação são uma representação eficiente do espaço navegável em um ambiente 3D. Elas consistem em polígonos que representam áreas onde um agente pode se mover.

5.4 Algoritmos de Busca Heurística

Um algoritmo de busca heurística

5.4.1 Algoritmo de Dijkstra

O pai dos algoritmos de busca.
Falhas.

5.4.2 Funções Heurísticas e Estimativas de Custo

Bússola. Função lambda.
Table com funções

5.4.3 A-Star (A*)

Dijkstra com Heurística.
Melhor de todos, monstruoso.

5.4.4 Jump Point Search (JPS)

Simplificação do A* para grafos em grid de peso uniforme.
Abstração para o 3D.

5.4.5 Theta-Star (Theta*)

5.5 Multithread

Importância do paralelismo nesses casos.
 Separação do contexto gráfico.
 Controle de threads.
 Gerenciamento controlado.

5.5.1 Gerenciamento de Tarefas com Monitor

Não passar dos limites do processador.
 Thread pool, senão overhead absurdo.
 RAII do C++.
 Mutex generalizado.
 jthreads e stop_tokens
 mutex e condicional_variable.
 Criação das threads workers e adição de task.
 Fila multinível.

5.5.2 Testes de Estresse

Cenários complexos e concorrência.

5.6 Processo de Coleta e Tratamento de Dados

Variáveis dependentes: tempo de processamento de CPU, picos de memória, qualidade do caminho encontrado (distância, continuidade), overheads de otimização, vértices percorridos.

Importância de se prender aos números.
 Análise estatística (95% de certeza).
 Valores estatísticos (taxa de variação, etc)

5.6.1 Definição de Variáveis e Métricas

Principais valores que impactam um algoritmo de busca.
 Coleta dos dados e alterações temporárias.

5.6.2 Pipeline de Exportação e Análise

Wrapper de coleta de dados.
 Separação por grupos, variáveis e dados.
 Análise estatística.
 Plotagem.

6 CRONOGRAMA

Tabela 1 – Cronograma de execução das atividades

ETAPAS	PERÍODOS
Revisão de Bibliografia	Jan/2026 a Fev/2026
Arquitetura Base (IFCG) e Estrutura de Grafos	Fev/2026 a Abr/2026
Heurísticas e Implementação Base (A*, Dijkstra, JPS, Theta*)	Mar/2026 a Mai/2026
Geração de Cenários de Teste e Malhas 3D	Abr/2026 a Mai/2026
Multithreading e Testes de Estresse	Mai/2026 a Jun/2026
Coleta de Dados Iniciais e Análise Estatística Simples	Mai/2026 a Jun/2026
Redação e Revisão do Pré-Projeto	Abr/2026 a Jun/2026
Defesa do Pré-Projeto	Jun/2026
Implementação de NavMeshes e Otimização 3D	Jul/2026 a Ago/2026
Otimização de Concorrência e Testes de Estresse	Ago/2026
Implementações Específicas (D* Lite, Busca Bidirecional, Lazy Theta*)	Ago/2026 a Set/2026
Coleta Automatizada de Métricas e Dados	Set/2026 a Out/2026
Análise Estatística e Visual dos Resultados	Out/2026 a Nov/2026
Redação da Monografia Final	Out/2026 a Nov/2026
Revisão e Entrega Final	Nov/2026
Defesa do TCC	Dez/2026

Fonte: Elaborado pelo autor (2026).

REFERÊNCIAS

BUCK, Jamis. **Maze Generation: Kruskal's algorithm**. Disponível em: <<https://weblog.jamisbuck.org/2011/1/3/maze-generation-kruskal-s-algorithm>>.

CORMEN, Thomas H. *et al.* **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro, RJ: Elsevier, 2012.

GAMMA, Erich *et al.* **Design Patterns: Elements of Reusable Object-Oriented Software**. [S.l.]: Addison-Wesley, 1994.

GOMES, Paulo César Rodacki. **Grafos: conceitos fundamentais, algoritmos e aplicações**. Blumenau, SC: Editora do Instituto Federal Catarinense, 2022.

GURUNG, Sabin. **Extension On Navigation Meshes**. Bachelor Thesis—Barcelona: Universitat Politècnica de Catalunya, 2019.

THE IMPERIAL LIBRARY. **Maps of Solstheim**. Disponível em: <<https://www.imperial-library.info/content/maps-solstheim>>.

PATEL, Amit J. **Making maps with noise functions**. Disponível em: <<https://www.redblobgames.com/maps/terrain-from-noise/#elevation-redistribution>>. Acesso em: 7 maio. 2026.

RUSSELL, Stuart J.; NORVIG, Peter. **Inteligência artificial**. 3. ed. Rio de Janeiro, RJ: Elsevier, 2013.

VRIES, Joey de. **Learn OpenGL: Enjoy learning modern OpenGL**. Disponível em: <<https://learnopengl.com/>>. Acesso em: 18 abr. 2026.

ZIPPED. **C++: Perlin Noise Tutorial**. Disponível em: <<https://www.youtube.com/watch?v=kCJaHqb60Cw>>. Acesso em: 7 maio. 2026.